

# Browser Extensions



# History

Not so long ago, Firefox and Internet Explorer ruled.

Extensions tended to interface closely with browser internals, so they were browser-specific and often required writing code in languages that web enthusiasts might find... unnatural.

```
using Microsoft.Win32;  
using mshtml;  
using SHDocVw;
```



```
namespace InternetExplorerExtension  
{  
    [ComVisible(true)]  
    [ClassInterface(ClassInterfaceType.None)]  
    [Guid("D40C654D-7C51-4EB3-95B2-1E23905C2A2D")]  
    [ProgId("MyBHO.WordHighlighter")]  
    public class WordHighlighterBHO : IObjectWithSite  
    {  
        const string DefaultTextToHighlight = "br";  
  
        IWebBrowser2 browser;  
        private object site;  
  
        #region Highlight Text  
        void OnDocumentComplete(object pDisp, ref  
        {  
            trv
```

Since the XPCOM layer itself is written in C/C++, its API can be accessed out-of-the-box using C or C++. In

The following bridging layers are currently available:



## [JavaXPCOM](#)

JavaXPCOM allows for communication between Java and [XPCOM](#), such that a Java application can access XPCOM objects, and XPCOM can access any Java class that implements an XPCOM interface. JavaXPCOM is not actively maintained.

## [PlXPCOM](#)

plXPCOM (Perl XPCOM) provides language bindings letting you use [XPCOM](#) from [Perl](#) code. The resources here provide information about this language binding and how to use it.

## [PyXPCOM](#)

PyXPCOM allows for communication between [Python](#) and [XPCOM](#), such that a Python application can access XPCOM objects, and XPCOM can access any Python class that implements an XPCOM interface. PyXPCOM is actively used in [ActiveState Komodo](#) products, for example.

# WebExtensions are born



Google Chrome came along and started dominating the market, and it had a well-defined, web-tech based extension API!

Eventually, this became the de-facto standard for browser extensions. Today all major browsers have support:

- Firefox added support in ~2015 (for various reasons\*)
- Safari added support in ~2021 (I believe - [WWDC20](#))
- Standardisation attempts:
  - [Browser Extension Community Group](#) (2015 - 2023)
  - [WebExtensions Community Group](#) (2021-)

\*Firefox was also about to introduce a bunch of major changes (multi-process, sandboxing, etc.), which would break many extensions built with the old APIs:

- <https://wiki.mozilla.org/WebExtensions>
- [https://hacks.mozilla.org/2015/09/lets\\_write\\_a\\_webextension/](https://hacks.mozilla.org/2015/09/lets_write_a_webextension/)
- [https://web.archive.org/web/20200429120245/https://developer.mozilla.org/en-US/docs/Archive/Add-ons/Working\\_with\\_multiprocess\\_Firefox](https://web.archive.org/web/20200429120245/https://developer.mozilla.org/en-US/docs/Archive/Add-ons/Working_with_multiprocess_Firefox)

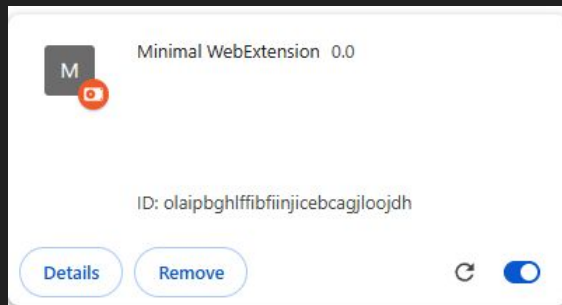
Chrome extensions logo: <https://web.archive.org/web/20091221045015/http://www.chromeextensions.org/>

# Defining a WebExtension - manifest.json

- The minimal WebExtension is a directory with a manifest.json file
- manifest.json describes the extension, including:
  - Name, description, version, icon, etc.
  - Configuration for functionality
  - Permissions

```
minimal  
  {} manifest.json
```

```
{  
  "manifest_version": 3,  
  "name": "Minimal WebExtension",  
  "version": "0.0"  
}
```



**Every event** has been updated. You must approve new permissions before the updated version will install. Choosing "Cancel" will maintain your current add-on version.

It requires your permission to:

- Exchange messages with programs other than Firefox
- Extend developer tools to access your data in open tabs
- Clear recent browsing history, cookies, and related data
- Read and modify browser settings
- Read the text of all open tabs
- Access your location
- Display notifications to you
- Control browser proxy settings
- Access browsing history

Cancel

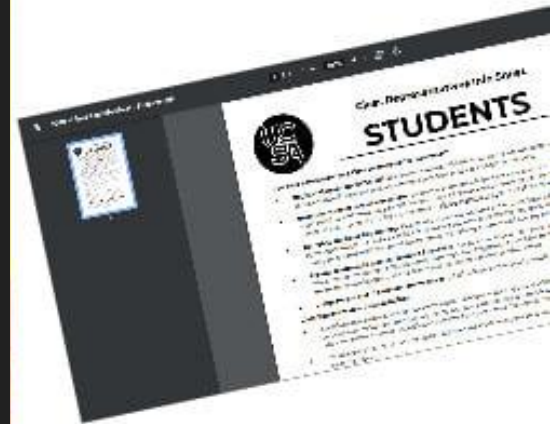
Update

# My first WebExtension

I have a problem:

- My university's student portal keeps displaying PDFs embedded in the page
- I want them to be full-screen
- I'm lazy

Solution: WebExtension time!



# The code

The embedded PDFs are <object> elements with the type set to “application/pdf”:

```
<object type="application/pdf" data="http://127.0.0.1:8000/download.pdf" width="500" height="500">...</object>
```

Here's some code which finds these and opens the PDF:

```
// opener.js
const __LEARN_PDF_opener = () => {
  const pageObjects = document.getElementsByTagName("object");
  for (const obj of pageObjects) {
    if (obj.type === "application/pdf" && obj.data.startsWith('http')) {
      window.location.replace(obj.data);
    }
  }
};

__LEARN_PDF_opener();
```

# Doing it automatically

At the moment that's just a code snippet.

To have the browser run it for us, we need to extend our manifest.json file:

```
{  
  "manifest_version": 2,  
  "name": "[Demo version] LEARN PDF auto-opener",  
  "version": "1.1",  
  "content_scripts": [  
    {  
      "matches": ["*://*.learn.canterbury.ac.nz/*", "*://127.0.0.1/*"],  
      "js": ["opener.js"]  
    }  
  ]  
}
```

Run it as a Content Script because we want to read/write the DOM

Only run it on certain pages

Our JavaScript snippet

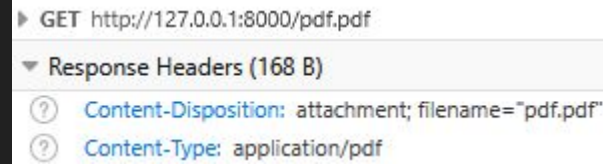
<Live demo />



# Another problem - forced downloads

Sometimes instead of embedding the PDF, the site forces me to download!

The server does this by adding a “Content-Disposition: attachment” response header. If only we could modify the response headers...



# Approach 1: Background Scripts and Blocking webRequest

The blocking webRequest API lets us arbitrarily rewrite response data with JavaScript.

1) Set up a background script + request permissions:

```
{
  "manifest_version": 2,
  "name": "[Demo version] LEARN PDF auto-opener with WebRequest",
  "version": "1.1",
  "content_scripts": [...],
  "background": {
    "scripts": [
      "background.js"
    ]
  },
  "permissions": [
    "webRequest", "webRequestBlocking",
    "*/**.*.learn.canterbury.ac.nz/*", "*/**127.0.0.1/*"
  ]
}
```

2) Attach a webRequest listener in our background script

```
const onHeadersReceived = (requestDetails) => { ...
}

chrome.webRequest.onHeadersReceived.addListener(
  onHeadersReceived,
  {
    urls: ["*/**.*.learn.canterbury.ac.nz/*", "*/**127.0.0.1/*"],
    types: ["main_frame"]
  },
  ["blocking", "responseHeaders"]
)
```

## Approach 2: declarativeNetRequest

Chrome is removing blocking webRequest from the new “Manifest v3” extension API they’re rolling out this year. Instead, we must use a declarative approach:

1) Enable declarativeNetRequest

```
"declarative_net_request": {  
  "rule_resources": [  
    {  
      "id": "learn_force_download",  
      "enabled": true,  
      "path": "dnr_rules.json"  
    }  
  ],  
},  
"permissions": [  
  "declarativeNetRequestWithHostAccess"  
],  
"host_permissions": [  
  "*/**/*.learn.canterbury.ac.nz/*", "*/**/127.0.0.1/*"  
]  
}
```

2) Set up a rule which overwrites the Content-Disposition header

```
[{  
  "id": 1,  
  "priority": 2,  
  "action": {  
    "type": "modifyHeaders",  
    "responseHeaders": [  
      {  
        "header": "Content-Disposition",  
        "operation": "set",  
        "value": "inline"  
      }  
    ]  
  },  
  "condition": {  
    "requestMethods": ["get"],  
    "urlFilter": "/*.pdf*",  
    "resourceTypes": ["main_frame"]  
  }  
}]
```

<Live demos />

# The Manifest v3 controversy

The decision to remove blocking webRequest has been hugely controversial!

- Ad blockers like uBlock Origin depend heavily on blocking webRequest
- declarativeNetRequest is not powerful enough to replace existing functionality built on blocking webRequest

Despite this, Chrome has started disabling Manifest v2 extensions automatically, and will remove support in Chrome 139 (to be released July/August 2025).

Firefox and Opera have said they'll continue to support blocking webRequest.

Brave has promised to “continue to support some privacy-relevant [manifest v2] extensions” “for as long as we’re able”.

Chrome Manifest v2 deprecation timeline: <https://developer.chrome.com/docs/extensions/develop/migrate/mv2-deprecation-timeline>

Chromium release schedule: <https://chromiumdash.appspot.com/schedule>

Firefox's position: <https://blog.mozilla.org/addons/2022/05/18/manifest-v3-in-firefox-recap-next-steps/>

Opera's position: <https://blogs.opera.com/news/2024/10/opera-support-manifest-v2-ad-blocking/>

Brave's position: <https://brave.com/blog/brave-shields-manifest-v3/>

# Many more things!

- There are heaps more things you can do with WebExtensions!
  - Extend the developer tools (like the React Devtools, Apollo devtools, etc.)
  - Create action popovers and pages
  - Customise context menus
  - Handle protocols
  - Call native applications
  - Interact with the clipboard
  - [And more!](#)
- Let's look at one more, which I think is particularly neat - native messaging!

## Native messaging

- Enables message passing between WebExtensions and native applications
- This is great for things like password managers (in fact this is how 1Password works!)
- Native applications must register themselves with the browser to 'opt in' to receiving messages from specific browser extensions, enabling some kind of security.

I implemented a simple password manager using native messaging and some of the other things we've looked at today, let's see how it works!

# Distribution

- It's quite easy!
- The process varies between browsers, but is generally a variation on:
  - Make a zip archive of your extension
  - Upload it to the browser's website
  - Fill out a form, and it's listed on their store
- Mozilla didn't want my extension on their store, so the process was:
  - Upload zip archive to the browser's website
  - Download a signed version of the zip archive
  - Self-host it alongside a special JSON file which Firefox can use to check for updates ([example](#))



# The world is your oyster

- There's a lot more
  - Check out the docs and tutorials - [MDN](#) / [Chrome](#)
- Have a go! For inspiration, think about what you would change in your browsing experience today:
  - Hyperlinking Jira tickets in PRs?
  - Customising the favicons of your stores?
  - Decoding binary UUIDs?
  - Filling out forms?
  - Running Docker commands?
  - Applying a sepia filter to everything?

Thanks!

